PY102

Insertion sort (vs. bubble)

Today
Welcome to your next lesson: Insertion sort (vs. bubble).

I'm preparing your personalized lesson based on what you've learned, what's coming up, and the best way for you to grow.

Step by step, you’re moving forward.

Ron, imagine you’re holding a hand of playing cards ♠♥

You start with 1 card, then you draw a new one and slide it into the right place among the cards you already hold, so the left side of your hand always stays sorted. You keep doing this: take the next card, compare it to the ones on the left, and insert it where it belongs.

That “growing sorted left side” is the core idea of insertion sort:

you repeatedly take the next element and insert it into its correct position among the elements to its left.

Does that picture with the cards make sense so far, or should I walk through a tiny numeric example like [5, 2, 4] step by step?

it makes sense

Nice 👍

I’ve just pulled up a short video that animates exactly that card-style insertion sort idea you understood.

Watch the video when you’re ready 👀

After you finish, tell me:

Do you want to:

go through a tiny step-by-step example by hand, or
jump straight to seeing the Python version of insertion sort?

Let's do the tiny example

Perfect, tiny example it is 😄

Let’s sort this list with insertion sort:

[5, 2, 4]

We think of it like this:

The left side is the “sorted hand” 🤚
The right side is the pile we still need to insert from.

Step by step:

Start: [5, 2, 4]

Left “hand”: [5] (already sorted, just one element)
Next card to insert: 2
Insert 2 into the sorted left side [5]

Compare 2 with 5 → 2 is smaller
So 2 slides in before 5
List becomes: [2, 5, 4]
Now left side is [2, 5] (sorted)
Next card to insert: 4

Compare 4 with 5 → 4 is smaller → move 5 one step right
Temporary: [2, 5, 5]
Compare 4 with 2 → 4 is bigger → stop here
Put 4 in the empty slot: [2, 4, 5]

Now the whole list is sorted ☑

Does this step-by-step flow match how you were picturing it, or is there any step you’d like me to slow down and re-draw in another small example?

Yes, that makes sense

Great 👍

Now I want to zoom in on what insertion sort is really doing, in one simple sentence:

You walk through the list from left to right, and for each element, you shift bigger elements to the right until you find the correct spot, then drop the element in there.

This keeps a sorted left part that grows one element at a time.

So the pattern is:

Pick the next value.
Look left while things are bigger.
Shift them right.
Insert the value into the “hole”.
Before we touch code, do you want to see this as a visual animation with a small array, or go straight to a Python code walkthrough of insertion_sort?

Show the visual animation

Take your time playing with the animation below 👇

Step through a few moves and watch how the left side stays sorted while each new element gets inserted.

When you’re done exploring it, tell me: does the “shift right then drop into the hole” motion feel clear, or is any part still fuzzy?

Insertion sort growing sorted left side

feels ckear

Nice, that’s exactly what I wanted 😊

Now let’s connect this to something you already know: bubble sort.

Very high-level:

Bubble sort: keeps swapping neighbors to push big elements step-by-step toward the end of the list (like bubbles rising up).
Insertion sort: keeps a sorted left side and shifts elements right to insert the current element into the correct spot.

So:

Bubble: many neighbor swaps
Insertion: shifts and a single insert per element
Quick check: if you had a list that was already almost sorted, which one do you expect to usually do less work:

the one that keeps swapping neighbors (bubble) or the one that just does a few shifts to insert (insertion)?

insertion

Exactly 💯 — insertion sort usually wins on almost-sorted data because it only does a few quick shifts for each element instead of lots of neighbor swaps.

Let’s make that concrete in Python now 🔧

I’ll show you the shape of the algorithm, then I’ll ask you to fill in a small piece yourself in the code editor.

Idea in code steps:

Start from index 1 (since index 0 alone is “sorted”).
Store the current value in a key variable.
Move left with an index j, shifting any elements greater than key one step to the right.
When you stop, put key into the empty spot.

Here’s a skeleton:

Python

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # TODO: shift larger elements to the right
        # while j >= 0 and arr[j] > key:
        #     ...
        #     j -= 1

        # TODO: put key into its correct position
        # arr[?] = key
```

Your turn 🫵

Open the code editor, paste this skeleton, and complete the two TODO parts so it

actually sorts the list in place. Then:

Test it with print(insertion_sort([5, 2, 4, 6, 1, 3])) (or print the list after calling it).

When you've submitted your code, I'll check it and we'll talk about why its time is $O(n^2)$ on average but closer to O(n) when the list is almost sorted.

Python

```
def insertion_sort(arr):
    """ Takes a list and sorts a copy of it susing insertion sort
        returns the sported list, leaving original intact
    """
    # Make a copy of the list
    tmp_list = arr.copy()
    # Loop through the list, 2nd item to last
    for pass_cnt in range(1, len(tmp_list)):
        # Store current item we want to sort into position
        key = tmp_list[pass_cnt]
        # setup to look left
        pos = pass_cnt - 1
        #  shift larger elements to the right
        while pos >= 0 and tmp_list[pos] > key:
            tmp_list[pos+1] = tmp_list[pos]
            pos -= 1
        # Insert our current term into the correct spot
        tmp_list[pos+1] =key
    return(tmp_list)
# Sorry for embelishment to leave it to end user to keep
# original list over overridge with
#   list = insertion_sort(list)
sorted_list = insertion_sort([5, 2, 4, 6, 1, 3])
print(sorted_list)
```

Ron, this is a really clean insertion sort implementation 👏

You:

correctly copied the list so the original isn't mutated
used key + pos exactly like the textbook version
got the shifting loop and pos+1 insert spot perfect
And the output [1, 2, 3, 4, 5, 6] confirms it works ☑
Now I want to focus on one idea: how much work this does.

Key fact about insertion sort:

On average, it does about $O(n^2)$ work, but if the list is almost sorted, it behaves closer to O(n) (just a few comparisons and almost no shifts).

To feel that, we can count comparisons instead of time.

Very small change: inside your while loop, add a count += 1, and print the count at the end, then:

Run it on something jumbled, like [5, 2, 4, 6, 1, 3]
Run it on something almost sorted, like [1, 2, 3, 4, 6, 5] or [1, 2, 3, 4, 5, 6]

Use a count variable (start at 0) to track how many times the while condition is checked.

Can you update your code in the editor to:

add a count for comparisons inside the while,
print the final count for each test list,

then submit it?

We'll compare the numbers and tie it directly to O(n²) vs "almost O(n)".

Python

```
def insertion_sort(arr):
    """ Takes a list and sorts a copy of it susing insertion sort
        returns the sported list, leaving original intact
    """
    count = 0
    # Make a copy of the list
    tmp_list = arr.copy()
    # Loop through the list, 2nd item to last
    for pass_cnt in range(1, len(tmp_list)):
        # Store current item we want to sort into position
        key = tmp_list[pass_cnt]
        # setup to look left
        pos = pass_cnt - 1
        #  shift larger elements to the right
        while pos >= 0 and tmp_list[pos] > key:
            tmp_list[pos+1] = tmp_list[pos]
            pos -= 1
            count += 1
        # Insert our current term into the correct spot
        tmp_list[pos+1] =key
    print(f'{count} passes')
    return(tmp_list)
# Sorry for embelishment to leave it to end user to keep
# original list over overridge with
#   list = insertion_sort(list)
lst1 = [5, 2, 4, 6, 1, 3]
lst2 = [1, 2, 3, 4, 6, 5]
sorted_list = insertion_sort(lst1)
print(sorted_list)
sorted_list = insertion_sort(lst2)
print(sorted_list)
```

Love this, Ron — this is exactly the experiment I was hoping you'd run 🔥

You just saw:

For lst1 = [5, 2, 4, 6, 1, 3] → 9 passes
For lst2 = [1, 2, 3, 4, 6, 5] (almost sorted) → 1 pass

Same length list, but very different work. That's the heart of insertion sort's behavior:

On random / jumbled lists → lots of shifting → about O(n²) work
On nearly sorted lists → almost no shifting → closer to O(n)

So if the data is already "pretty good", insertion sort barely has to move anything.

Now let's connect this back to bubble sort you just learned:

Bubble sort keeps doing neighbor swaps across the list, pass after pass.
Insertion sort builds a sorted left side, doing a few shifts per element.

Thinking just at a high level, for big lists, how would you compare them?

Which statement sounds more accurate to you:

"Bubble sort is usually faster than insertion sort on almost-sorted data"
"Insertion sort is usually faster than bubble sort on almost-sorted data"
"They're exactly the same in all cases"

You can just answer in words; we'll then do a tiny wrap-up on their differences.

"Insertion sort is usually faster than bubble sort on almost-sorted data"

Exactly right 🎯

That matches the big idea:

Both bubble sort and insertion sort are $O(n^2)$ for large, messy lists.
But when the list is already close to sorted, insertion sort usually does much less work than bubble sort, because:
it stops shifting as soon as it finds the right spot,
while bubble still walks through neighbor pairs doing lots of comparisons and swaps.

So you've hit all three key points for this lesson:

What insertion sort does:
Repeatedly takes the next element and inserts it into its correct position in the growing sorted left side, by shifting bigger elements right.

Its time behavior:

Typical / random data → about $O(n^2)$ work
Nearly sorted data → much closer to $O(n)$, as you just measured with your count.
Compared to bubble sort:

Both are $O(n^2)$ overall.
Bubble: lots of neighbor swaps, "bubbling" big values to the end.
Insertion: shifts and inserts, usually better on almost-sorted data.
Let's do one quick check to lock it in 🔓

Imagine three lists, all length 8:

A: [1, 2, 3, 4, 5, 6, 7, 8] (already sorted)
B: [2, 1, 3, 4, 5, 6, 7, 8] (just the first two are swapped)
C: [8, 7, 6, 5, 4, 3, 2, 1] (reverse order)

Which one do you think insertion sort will finish fastest on? You can just answer with A, B, or C.

A: almost no work (just one time tthrought to see its sortd) just say tada! Done

What do you think? Pick the option that best matches how insertion sort works 👇

Which description best matches insertion sort?

A.

Repeatedly find the smallest element in the whole list and swap it with the first position.

B.

Repeatedly take the next element and insert it into the correct position among the elements to its left, shifting bigger ones right.

C.

Repeatedly swap adjacent out-of-order pairs to bubble the largest element toward the end.

D.

Randomly shuffle the list until it becomes sorted.

Yes! 🎉 You picked the perfect description:

Insertion sort = repeatedly take the next element and insert it into the correct position among the elements to its left, shifting bigger ones right.

That matches exactly what you:

acted out with the card-hand analogy, and
implemented in Python with your key + shifting loop.
The "growing sorted left side" + "shift bigger ones right, then drop the key in the hole" is the core mental model — and you clearly own it now 💪

You've nailed:

how insertion sort works,
why it's O(n²) on average but close to O(n) on nearly sorted data,
and how it compares to bubble sort.
Nice work today, Ron.
When you're ready, you can jump into the next lesson in the Learn tab 🚀

```
def insertion_sort(arr):
    """ Takes a list and sorts a copy of it susing insertion sort
        returns the sported list, leaving original intact
    """
    count = 0
    # Make a copy of the list
    tmp_list = arr.copy()
    # Loop through the list, 2nd item to last
    for pass_cnt in range(1, len(tmp_list)):
        # Store current item we want to sort into position
        key = tmp_list[pass_cnt]
        # setup to look left
        pos = pass_cnt - 1
        #  shift larger elements to the right
        while pos >= 0 and tmp_list[pos] > key:
            tmp_list[pos+1] = tmp_list[pos]
            pos -= 1
            count += 1
        # Insert our current term into the correct spot
        tmp_list[pos+1] =key
    print(f'{count} passes')
    return(tmp_list)
# Sorry for embelishment to leave it to end user to keep
# original list over overridge with
#   list = insertion_sort(list)
lst1 = [5, 2, 4, 6, 1, 3]
lst2 = [1, 2, 3, 4, 6, 5]
sorted_list = insertion_sort(lst1)
print(sorted_list)
sorted_list = insertion_sort(lst2)
print(sorted_list)
```